Enterprise Audit Trails

who did what, to what, when, and from where

Watch the 4-min video

In one line: an audit trail is a product feature: structured, tenant-scoped records of every security-relevant action, written through a pipeline that can't lose them, stored **append-only and tamper-evident**, and searchable, exportable and streamable by the customer.

1 The incident question

An enterprise customer's security team is investigating an incident: *who exported our customer list last Tuesday, from which IP, and who changed the admin's permissions an hour before?* If your answer is "let me grep our server logs", you've lost their trust, and probably your SOC 2 auditor's. Enterprise customers expect to answer that question themselves, in minutes, from your product.

2 What an audit entry records

```
{ "id": "aud_01JA…", "tenant_id": "acme", "seq": 48213,
  "ts": "2026-11-03T02:07:41Z",
  "actor": { "type": "user", "id": "u_731", "email": "c.lee@contractor.io",
             "via": "sso", "impersonated_by": null },
  "action": "customer_list.exported", "result": "success",
  "target": { "type": "report", "id": "rep_customers" },
  "ip": "203.0.113.45", "user_agent": "Mozilla/5.0 …", "request_id": "r-9f2…",
  "changes": null,                 // {before, after} for updates
  "prev_hash": "a41c…", "hash": "7be0…" }
```

Figure 1. One audit event: actor, action, target, time, origin, result, and for changes the before and after values.

The core is **who** (actor: user, API key, service, or your own staff, with how they authenticated and whether someone was impersonating them), **did what** (a stable action name from a catalog, like `role.changed`, `report.exported`, `sso.config.updated`), **to what** (target type and ID), **when** (UTC timestamp from a synced clock), **from where** (IP, user agent, request ID for correlation) and **with what result** (success, denied, failed: failed logins and denied attempts are often the most interesting). Record `before` / `after` for changes, but never secrets: log that an API key was created, not the key.

What to cover: sign-in and sign-out, failed logins, MFA changes, user and role changes, SSO and SCIM events (lesson 47), permission and sharing changes, data exports and bulk downloads, API key lifecycle, settings and security policy changes, and every access by your own staff.

3 Audit logs are not debug logs

	Debug / application logs	Audit logs
Audience	your engineers	customers, security teams, auditors
Content	free text, noisy, anything	structured schema, defined action catalog
Completeness	sampled, rate-limited, best effort	every security-relevant action, never sampled
Retention	days to weeks	1 to 7+ years, per tenant
Mutability	rotated and deleted freely	append-only, tamper-evident
Access	internal tooling	tenant-scoped UI, API, SIEM stream

So audit events go through a **dedicated pipeline**. The classic gap is "the change committed but the audit event was lost". Close it with a **transactional outbox**: write the audit row in the same database transaction as the change, and let a relay publish it. For the most sensitive actions (exports, permission changes), **fail closed**: if the audit write fails, the action fails.

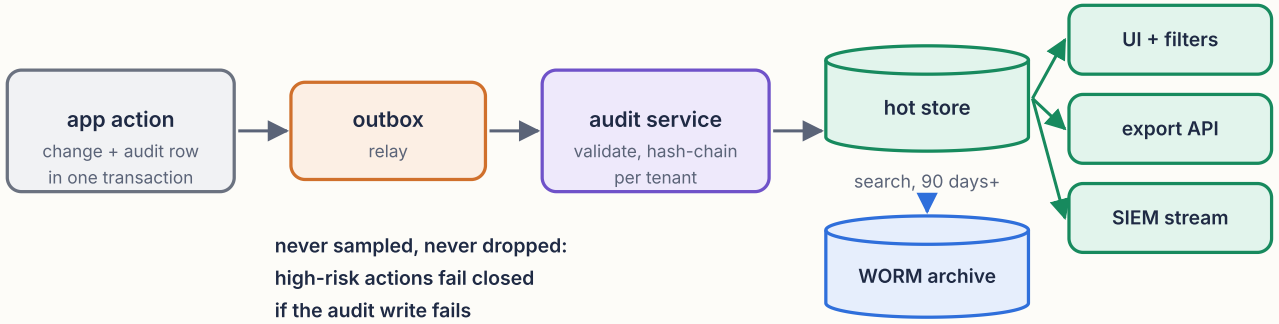


Figure 2. Audit pipeline: transactional capture, per-tenant chaining, hot searchable store, write-once archive, and three ways out for the customer.

4 Tamper evidence

An attacker's first move is often to hide their tracks, and insiders exist. Three layers:

- **Append-only access.** The application role can insert audit rows but not update or delete them; no human has write access in production.
- **Write-once storage (WORM).** Amazon S3 Object Lock in compliance mode blocks deletion or overwrite, even by the root account, until the retention date; Azure Blob immutable storage with a locked time-based retention policy does the same.
- **Hash chaining.** Each entry stores the hash of the previous one, per tenant: $h_n = \text{SHA-256}(h_{n-1} \,\|\, \text{entry}_n)$. Deleting or editing any entry breaks every link after it, which a verifier detects.

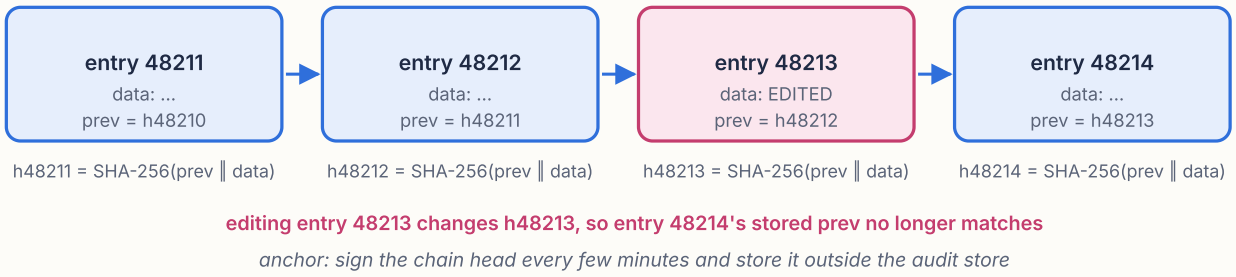


Figure 3. A hash chain makes changes detectable. Anchoring the head stops an attacker from simply recomputing the whole chain.

A chain alone has one hole: someone who can rewrite the whole store can recompute every hash. So periodically **anchor** the head: sign it with a key the audit service doesn't hold, and store the signed checkpoint somewhere else (another account, the WORM archive, or give it to the customer). Per-tenant chains also let you hand a customer a verifiable export of just their entries.

5 Customer access, retention and compliance

Customers need three ways in: an **audit page** with filters (actor, action, target, time range, IP), an **export API** (paginated JSON or CSV), and **streaming to their SIEM**, such as Splunk (via its HTTP Event Collector) or Microsoft Sentinel. Every query is scoped to the tenant (lesson 43). Retention is set per tenant: for example one year by default and seven years for regulated customers, with old entries moved from the hot store to the archive.

Frameworks expect this. SOC 2 auditors ask how you log access to customer data, how long you keep logs, who can change them and how you review them; ISO 27001:2022 covers it in Annex A control 8.15 (Logging); PCI DSS requires at least 12 months of audit history with the last three months immediately available. Your own staff's access, especially break-glass access, must appear in the same tenant-visible trail. Audit logs hold personal data (emails, IPs), so GDPR still applies: define retention, and restrict who can read them.

WORKED EXAMPLE: A CUSTOMER ADMIN EXPORTS HER AUDIT LOG

The admin filters the audit page to last week and action `customer_list.exported`. One row stands out: a contractor exported the full customer list at 02:07 from an IP never seen before. Filtering by that actor shows a role change to **admin** an hour earlier, made by an account that had a failed-MFA streak. She exports the week as JSON through the API, with each entry's hash so her team can verify the chain, and forwards it to Sentinel. Her export itself becomes a new audit event: `audit_log.exported`.

COMMON MISTAKES

- Mixing audit events into application logs, where sampling and rotation drop them.
- Writing the audit event after the commit, outside the transaction, so crashes lose it.
- Hiding your own staff's access from the customer's trail.
- Logging secrets or full payloads in before/after values.

6 Key terms

Audit trail: tenant-visible record of security-relevant actions

Actor / action / target: who did what to which resource

Hash chain: each entry stores the previous entry's hash

SIEM: security event platform, e.g. Splunk, Sentinel

WORM: write once, read many storage

Anchoring: signing and storing the chain head elsewhere

7 Interview questions

Q1. What do enterprise customers expect from audit logs? BASIC

Tenant-scoped records of who did what, to what, when and from where for all security-relevant actions; tamper-evident storage; search, export APIs and SIEM streaming; and configurable retention.

Q2. How do you make audit logs tamper-evident? BASIC

Append-only, write-once storage with hash-chained entries, so any change breaks the chain, plus periodically signed chain heads stored elsewhere.

Q3. How are audit logs different from debug logs? BASIC

They're a customer-facing, structured, complete and immutable record kept for years; debug logs are internal, noisy, sampled and short-lived.

Q4. How do you guarantee an audit event exists for every committed change? SCENARIO

Write it in the same database transaction as the change (transactional outbox) and fail closed for sensitive actions if the audit write fails.

Q5. A regulated customer needs seven years of audit logs. How do you store them affordably? SCENARIO

Keep recent months in a searchable hot store and move older entries to compressed, write-once object storage with a per-tenant retention policy.

Q6. Hash chaining is enough to stop an attacker with full database access, right? TRICKY

No: they can recompute the whole chain. Anchor the chain head by signing it with a separate key and storing checkpoints outside their reach.

Q7. Should your support engineers' access appear in the customer's audit log? TRICKY

Yes: staff access, especially break-glass, is exactly what customers and auditors want to see.

Next: SD-R-009 · Tenant Configuration backendroadmap.com